

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования «Тульский
государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине
Интеграция и тестирование программных систем
по теме
«Тестирование API как основа интеграции (часть 1)»

Выполнил: ст. гр. 220032-11

Лаукерт К.М.

Принял:

Басалов Ю.А.

Тула 2026

Лабораторная работа №2

по теме «Тестирование API как основа интеграции (часть 1)»

Цель работы

Получить практические навыки ручного тестирования REST API интерфейсов интеграции. Освоить работу с инструментом Postman для отправки запросов, анализа ответов и организации коллекций тестов; научиться проектировать позитивные и негативные тестовые сценарии, использовать переменные окружения и параметризацию, писать скрипты проверок (Tests) на JavaScript.

Описание тестового API и его интеграционных точек

Объект тестирования — JokeAPI (вариант 9): публичный REST API случайных шуток. Базовый URL: <https://v2.jokeapi.dev>.

Документация: <https://jokeapi.dev/>. API не требует авторизации, ответы возвращаются в формате JSON.

Интеграционная точка	Назначение
GET /joke/Any	Случайная шутка любой категории (позитивный сценарий по заданию)
GET /joke/Programming	Случайная шутка категории Programming (интеграционный риск — «неверная категория»)
GET /info	Метаданные API: версия, список категорий, флагов, диапазоны id
Параметры запросов	amount (1–10) — количество шуток; idRange — выборка по id; blacklistFlags — фильтр флагов контента

Интеграционный риск по варианту — «неверная категория»: при интеграции клиент может передать несуществующую категорию (опечатка, устаревший список), поэтому обязательны проверки как корректной категории, так и обработки несуществующей.

Подготовка рабочей среды

Установлен Postman (v12.30). Создана коллекция «JokeAPI Lab2» с группировкой запросов по типам операций: папки Positive (5 запросов) и Negative (7 запросов) — 12 тестовых сценариев.

Настроено окружение (Environment) «JokeAPI Env» с переменной `base_url = https://v2.jokeapi.dev`, используемой во всех запросах через `{{base_url}}`.

Параметризация: послеResponse-скрипт запроса API-02-01 извлекает id полученной шутки и сохраняет его в переменную коллекции joke_id (pm.collectionVariables.set); следующий запрос API-02-05 использует {{joke_id}} в параметре idRange и проверяет, что возвращается шутка с тем же id. Таким образом реализована цепочка «id из ответа → следующий запрос».

Таблица выполненных тестовых сценариев

Все 12 сценариев выполнены в Postman; для каждого зафиксированы ожидаемый и фактический результат (код состояния, тело ответа, результаты Tests-скриптов):

ID	Эндпоинт, метод	Параметры / тело	Ожидаемый результат	Фактический результат	Статус
API-02-01	GET /joke/Any	—	200, JSON-объект шутки: error=false, category, id (число)	200 ОК; шутка с error=false; тесты 4/4; id сохранён в joke_id	Pass
API-02-02	GET /joke/Programming	—	200, category = Programming (риск «неверная категория»)	200 ОК, category = Programming; тесты 5/5	Pass
API-02-03	GET /joke/Programming	amount=3	200, массив из 3 шуток, все Programming	200 ОК, amount=3, 3 шутки Programming; тесты 4/4	Pass
API-02-04	GET /info	—	200, метаданные: version, список категорий	200 ОК, version 2.3.3, Programming в списке; тесты 3/3	Pass
API-02-05	GET /joke/Any	idRange={{joke_id}}	200, шутка с id из ответа API-02-01 (параметризация)	200 ОК, id совпадает с joke_id; тесты 2/2	Pass
API-02-06	GET /joke/FooBar	—	4xx, error=true, сообщение о неверной категории	400, код 106, additionalInfo перечисляет допустимые категории; тесты 4/4	Pass
API-02-07	GET /joke/Programming	amount=11	400 Bad Request (превышение максимума amount=10)	200 ОК — amount молчаливо усечён до 10; тесты 0/2	Fail
API-02-08	GET /joke/Any	amount=-5	400 Bad Request (менее минимума amount=1)	200 ОК — одиночная шутка, параметр проигнорирован; тесты 0/2	Fail
API-02-09	GET /joke/Programming	blacklistFlags=nsfw,notAFlag	400, error=true, сообщение о неверном флаге	400, код 106, additionalInfo перечисляет допустимые флаги; тесты 3/3	Pass
API-02-10	GET /joke/Programming	idRange=999999-99999999	4xx, error=true (несуществующий id)	400, код 106 «No matching joke found»; тесты 3/3	Pass
API-02-11	GET /nonexistent	—	404, JSON-тело ошибки	404, тело — HTML-страница, а не JSON; тесты 1/2	Pass (с замечанием)

API-02-12	POST /joke/Any	—	405 Method Not Allowed (метод не поддерживается)	400, HTML-страница ошибки; тесты 0/1	Fail (с замечанием)
-----------	----------------	---	--	--------------------------------------	---------------------

Итог: 9 сценариев Pass (в т. ч. один с замечанием), 3 сценария Fail (в т. ч. один с замечанием) — выявленные особенности валидации JokeAPI (описаны в выводах).

Скриншоты ключевых запросов из Postman

Ниже приведены скриншоты ключевых позитивных и негативных запросов (полное окно Postman: слева — коллекция, в центре — запрос, тесты и ответ).

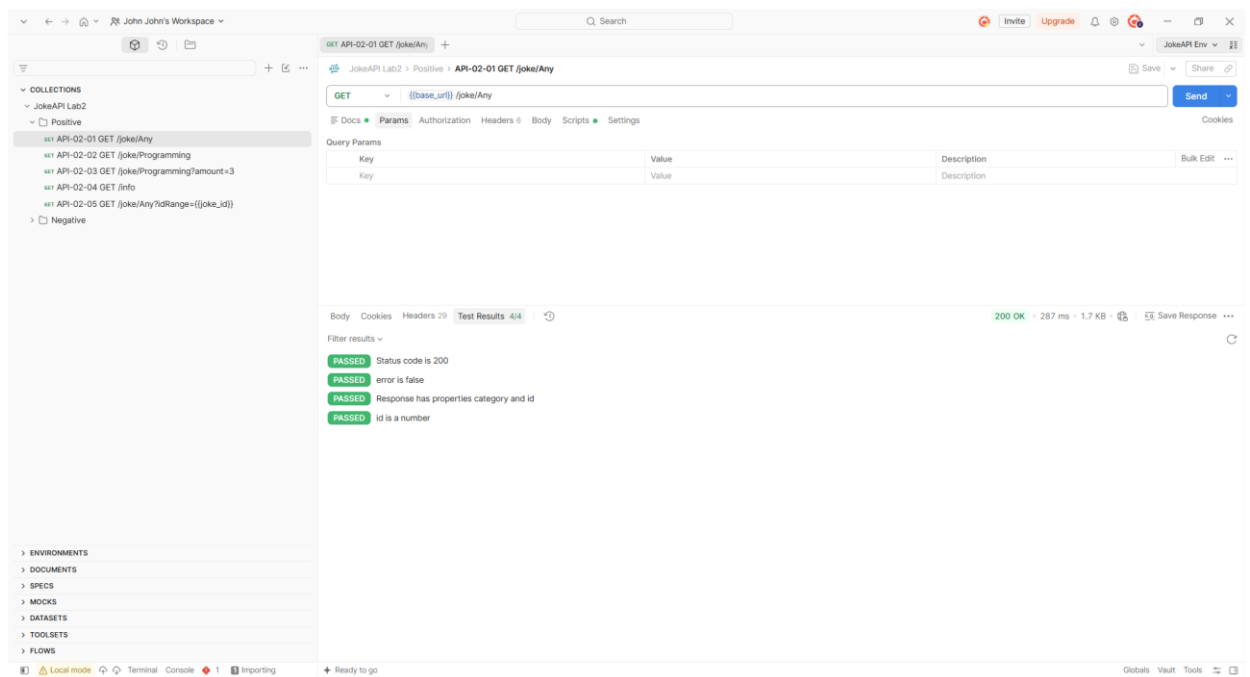


Рис. 1. API-02-01 GET /joke/Any: 200 OK, Test Results 4/4 (id сохранён в переменную `joke_id`)

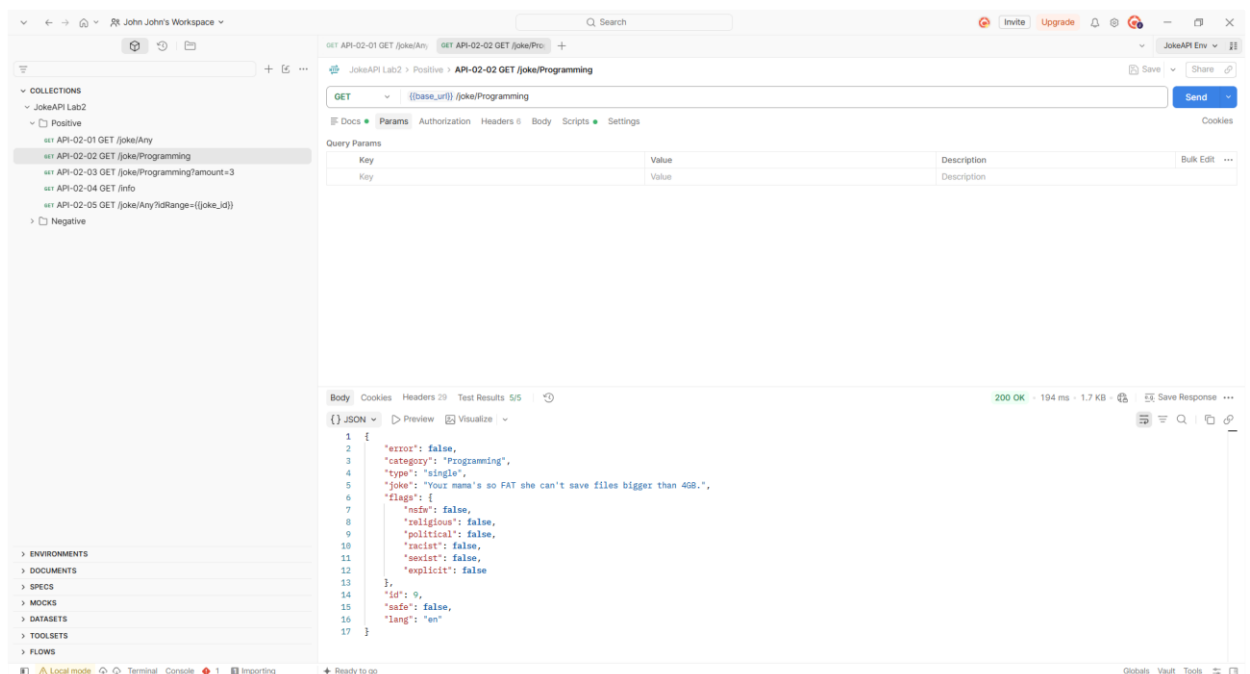


Рис. 2. API-02-02 GET /joke/Programming: 200 OK, category = Programming

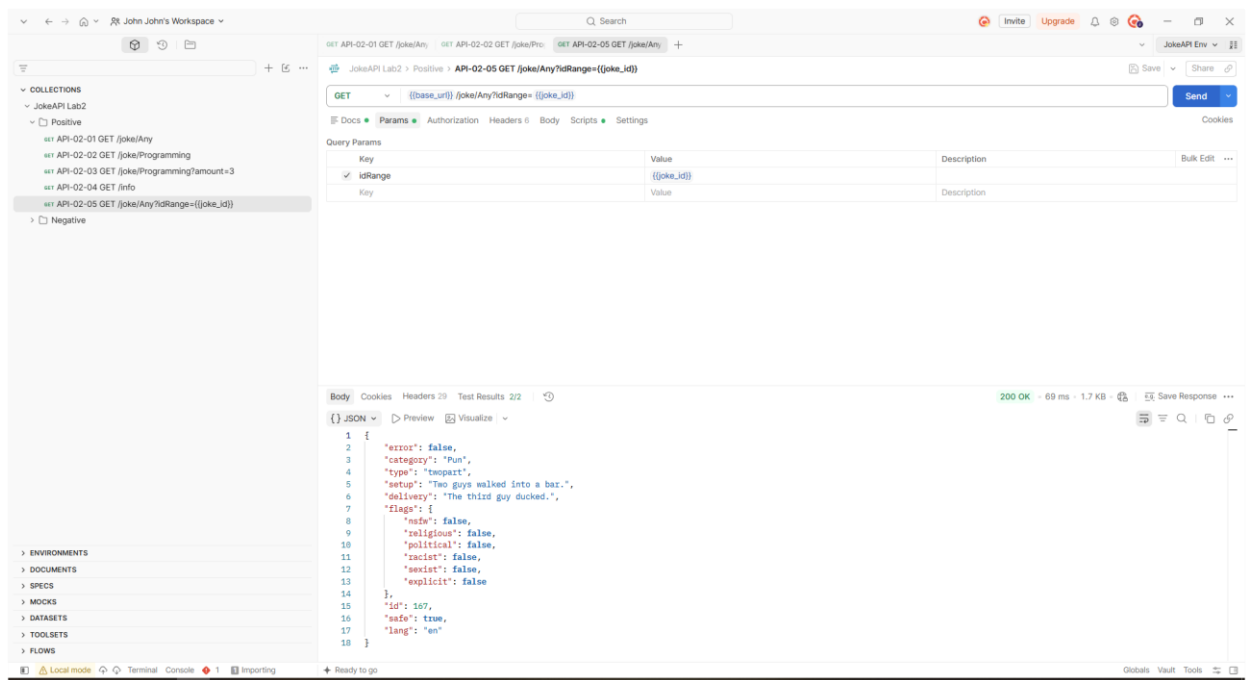


Рис. 3. API-02-05 GET /joke/Any?idRange={{joke_id}}: параметризация — из ответа получена шутка с тем же id

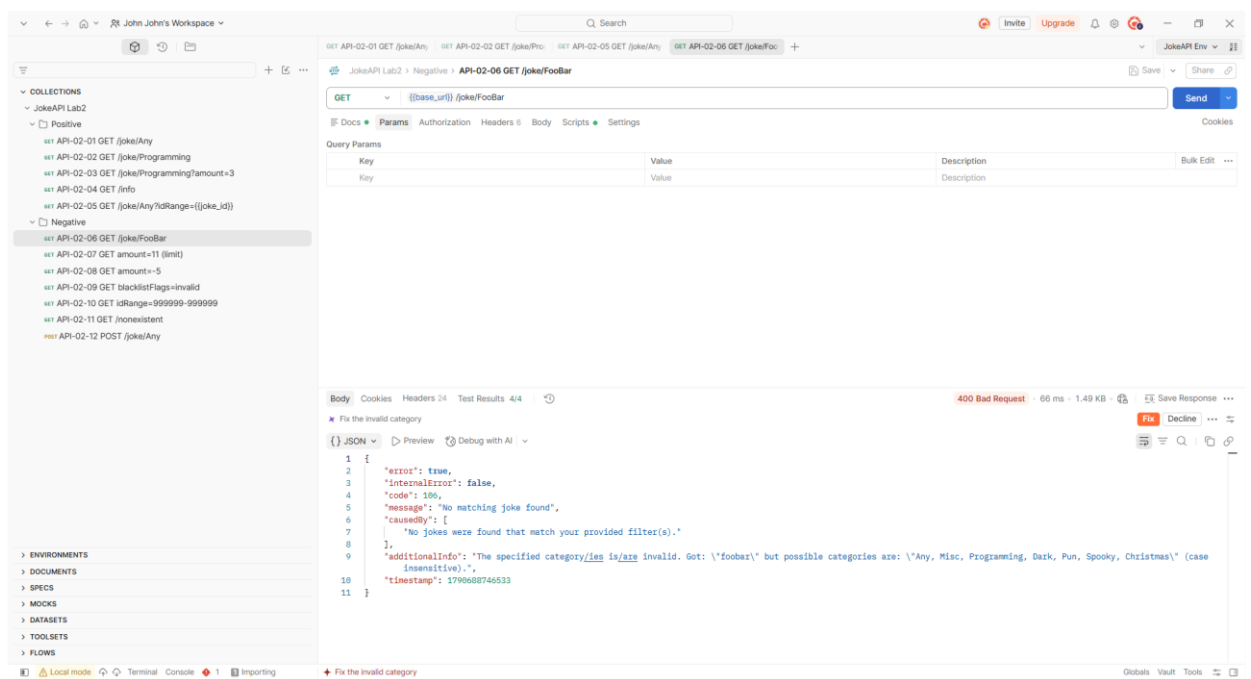


Рис. 4. API-02-06 GET /joke/FooBar (негативный): 400, JSON с error=true и кодом 106

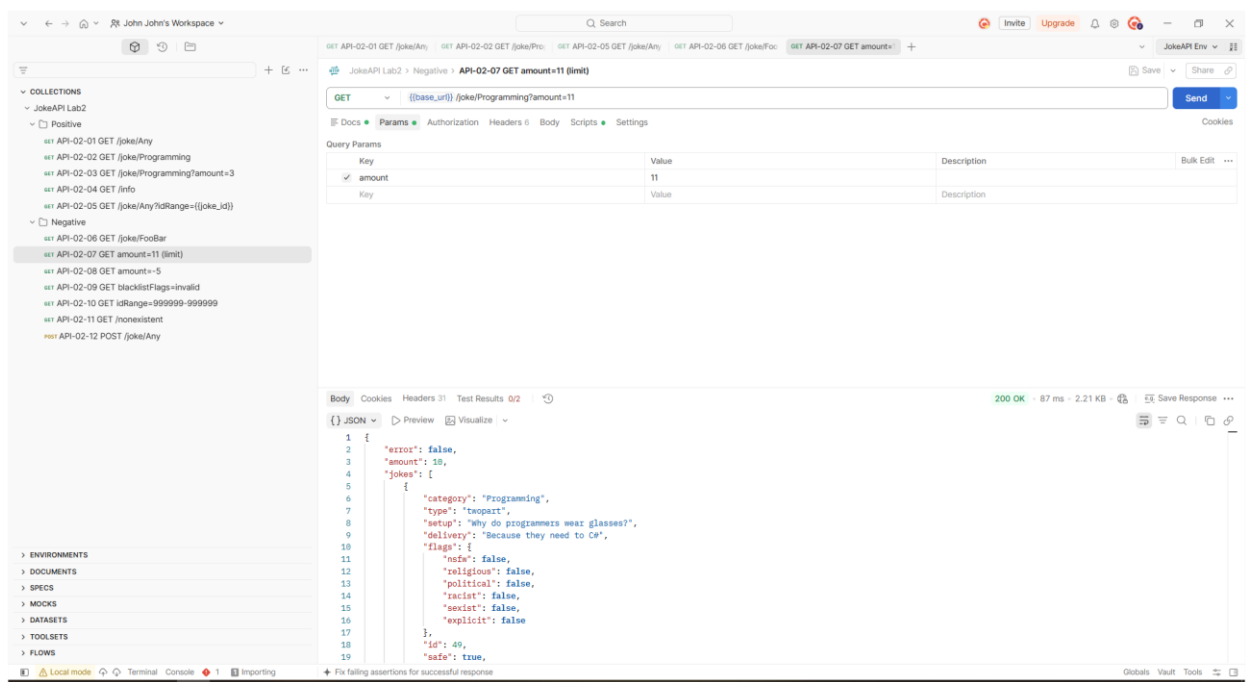


Рис. 5. API-02-07 amount=11 (негативный): 200 с усечением до 10, Test Results 0/2 — сценарий Fail

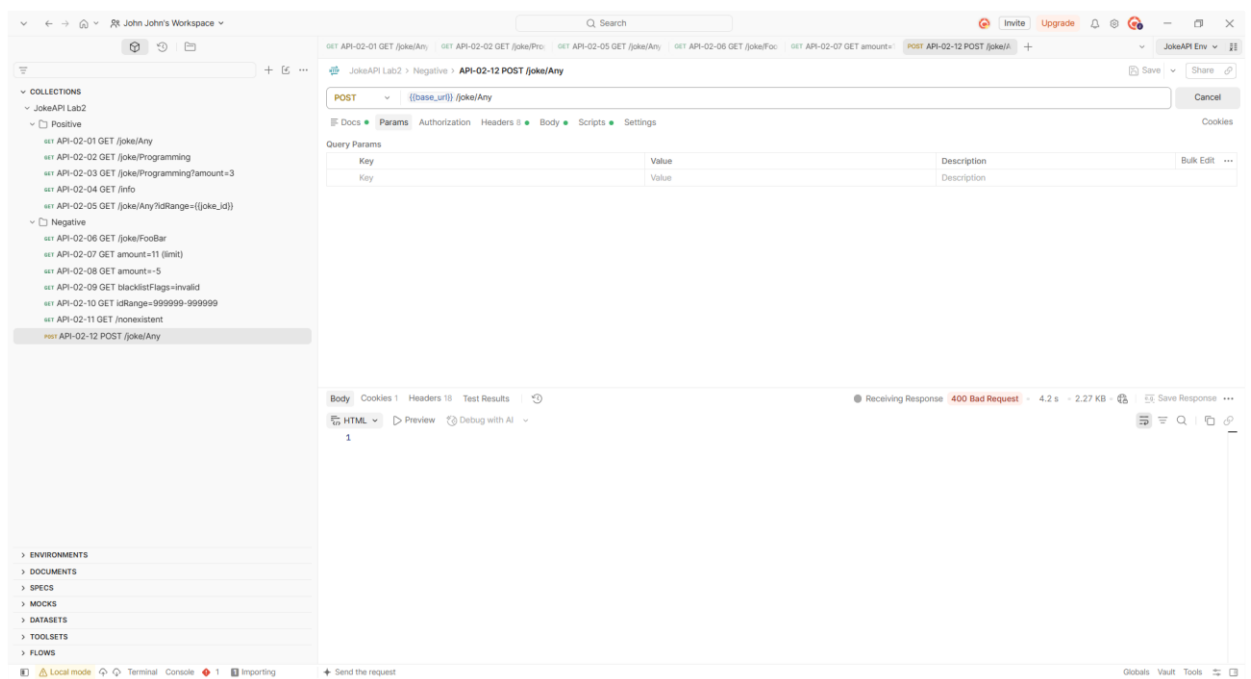


Рис. 6. API-02-12 POST /joke/Any (негативный): 400 Bad Request с HTML-телом вместо JSON

Пример кода скрипта проверки (Tests)

Позитивный сценарий API-02-01 — послеResponse-скрипт (вкладка Scripts → Post-response) с проверками по шаблону из задания и сохранением id в переменную коллекции:

```
const resp = pm.response.json();

pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("error is false", function () {
    pm.expect(resp.error).to.eql(false);
});

pm.test("Response has properties category and id", function () {
    pm.expect(resp).to.have.property("category");
    pm.expect(resp).to.have.property("id");
});

pm.test("id is a number", function () {
    pm.expect(resp.id).to.be.a("number");
});

pm.collectionVariables.set("joke_id", resp.id);
```

Негативный сценарий API-02-06 — проверка структуры ошибки:

```
const resp = pm.response.json();

pm.test("Status code is 400", function () {
    pm.response.to.have.status(400);
});

pm.test("error is true", function () {
    pm.expect(resp.error).to.eql(true);
});

pm.test("Response has property message", function () {
    pm.expect(resp).to.have.property("message");
});

pm.test("Error code is 106", function () {
    pm.expect(resp.code).to.eql(106);
});
```

Выводы

В ходе лабораторной работы протестировано 12 интеграционных сценариев публичного REST API JokeAPI в Postman: 5 позитивных и 7 негативных. Для всех ключевых сценариев написаны Tests-скрипты (`pm.response.to.have.status`, `pm.expect(...).to.have.property`, `pm.expect(...).to.be.a("number")`); налажены переменные окружения (`base_url`) и параметризация «id из ответа → следующий запрос» (`joke_id` → `idRange`). Корректность работы API по основным точкам подтверждена: валидные запросы возвращают 200 с корректной структурой JSON; интеграционный риск «неверная категория» обрабатывается корректно (400, код 106, `additionalInfo` со списком допустимых категорий). Выявлены особенности (проблемы) валидации, о которых должен знать интегрирующийся клиент: 1) превышение лимита `amount` (11) не отвергается — значение молчаливо усекается до 10 (API-02-07, Fail); 2) значение `amount` меньше минимума (-5) игнорируется и возвращается одиночная шутка (API-02-08, Fail); 3) для несуществующих путей и неподдерживаемых методов (POST) API возвращает HTML-страницу ошибки вместо JSON и код 400 вместо 405 — несогласованность формата ошибок, представляющая риск для клиентов, ожидающих JSON. Качество документации в целом хорошее (описаны параметры, лимиты, коды ошибок), однако поведение «мягкой» валидации `amount` и формат ошибок вне эндпоинтов `/joke` в ней явно не специфицированы — эти случаи следует закреплять контрактными тестами на стороне клиента.